Student Project Reflection: Cloud Knowledge Distillation & Deployment

Student Name: Nemapu Navaneeth	**Roll Number:** 230101086
Course: CS3081 Cloud Computing	**GitHub:** https://github.com/NavaneethNemapu/cloud-command-parser

1. Project Overview & Motivation

Running multi-billion parameter language models on low-end laptops is completely impractical due to heavy memory and compute requirements. In this project, I used sequence-level knowledge distillation (Kim & Rush, 2016) to compress a 3.8B teacher model (Phi-3.5-mini-instruct) into an ultra-compact 70M student model (EleutherAI/pythia-70m). I then built a production-grade cloud deployment on a local k3d Kubernetes cluster with SeaweedFS S3 storage, container resource limits (1 vCPU, 1 GiB RAM), horizontal autoscaling, stateless network policies (NACLs), and automated CI/CD testing.

2. What I Learnt (Core Technical Learnings)

• **Knowledge Distillation in Practice:** Sequence-level distillation avoids the heavy computational burden of matching 50,000+ vocabulary logits at every token position. Instead, the student trains directly on the teacher's greedily decoded text outputs using standard cross-entropy loss, successfully learning domain-specific JSON structure without needing GPU hardware.

• **CPU Hardware & Autograd Bottlenecks:** Deep learning backpropagation on CPU is severely bound by memory bandwidth. In causal models, the output unembedding layer (`embed_out`) causes heavy matrix multiplication thrashing unless batch sizes, sequence lengths, and CPU threads are strictly constrained.

• **Decoupled Object Storage:** Using SeaweedFS taught me how cloud-native object stores decouple metadata management (LevelDB Filer) from physical chunk storage (Volume servers). I implemented atomic directory locking (`os.mkdir`) as a filesystem mutex to prevent race conditions during worker data generation.

• **Kubernetes Probes & Cloud Lifecycles:** I learned the operational importance of separating liveness (`/healthz`) from readiness (`/readyz`) probes, ensuring that incoming traffic is blocked while downloading model weights from S3 during cold starts.

3. Major Engineering Challenges & Implemented Solutions

Challenge 1: The 15-Minute CPU Training Step Bottleneck

- *Problem:* Initially, fine-tuning with batch size 50 caused each training step to take over 15 minutes on CPU.
- *Diagnostic:* PyTorch autograd profiling revealed that **99.48% of CPU backward time** was spent in `aten::mm`, with **79.19% inside MmBackward0** (the 50,304 × 512 vocabulary projection layer), creating ~250 GFLOPs of dense computation per batch.
- *Solution:* I lowered batch size to 4, capped max sequence length to 48 with dynamic padding, and tuned intra-op threads to 4. This reduced active tokens per step by 12.5x and cut step execution time to ~28.9 seconds.

Challenge 2: SeaweedFS Master Volume Saturation

- *Problem:* Uploading trained weights to S3 failed with *'No writable volumes left'* errors.
- *Diagnostic:* SeaweedFS started with default `volume.max=8`, which became saturated by default and dataset collections, leaving zero free slots for the new `models` collection.
- *Solution:* I restarted the master with `"-volume.max=50"` and cleaned empty volumes with `volume.deleteEmpty`, creating 42 free volume slots.

Challenge 3: Stateless NACL Packet Dropping on Ephemeral Return Ports

- *Problem:* Simulated client requests and backend traffic were blocked by the network ACL.
- *Diagnostic:* A stateless NACL does not track connection state. Responses from HTTPS port 443 travel outbound to random client ephemeral ports (1024–65535). Without an explicit rule, return packets are dropped.
- *Solution:* I configured bidirectional rules permitting ephemeral return ports (1024–65535) and placed the deny rule for abusive subnet `198.51.100.0/24` at Rule 100 before general allow rules.

Challenge 4: Gradient Explosion & NaN Loss

- *Problem:* Small-batch training with a 5e-4 learning rate caused gradient divergence and NaN loss on batch 2.
- *Solution:* I lowered the learning rate to `0.0001` (1e-4) and added gradient norm clipping (`clip_grad_norm_(1.0)`), ensuring smooth monotonic loss convergence from 3.6991 to 1.5500.

4. Key Empirical Findings & Observations

• **Compression vs. Quality:** Distilling Phi-3.5 into Pythia-70M reduced parameters by **97.7%** (3.8B to 70M). Despite this reduction, the student retained ~95% of teacher task capability, achieving **97.0% JSON validity** and **59.0% full-frame accuracy** on validation data (well above the 31.0% pass threshold).
• **Resource Headroom:** Under 1 vCPU and 1 GiB limits, the service peaked at **554.00 MiB RAM** over 50 requests, preserving **45.9% safe headroom** below the 1,024 MiB ceiling with 0 OOM errors.
• **Cold Start:** S3 model retrieval and initialization took a **median of 9.26 seconds** across 3 independent runs (3.1s S3 transfer + 6.1s weight loading).
• **Production Load Test:** Over a 2-minute Locust benchmark (4 users, 3,264 requests), the deployment sustained **27.23 req/s** with **130 ms median latency**, **250 ms p95 latency**, and an exact **0.0% error rate**.
• **High Reliability:** Rolling updates (v1 to v2) and rollback undo executed under active load with 0 dropped requests.

5. Proposed Improvements (What Could Have Been Done Better)

1. **Token-Level Logit Distillation:** Combine sequence distillation with KL divergence over top-k teacher logits and hidden layer projections (FitNets) to improve slot boundary precision.
2. **INT8 Quantization & ONNX Runtime:** Quantize Pythia-70M weights from FP32 to INT8 to reduce disk size from 280 MB to ~70 MB and cut inference latency to <20 ms.
3. **Multi-Node High Availability:** Move beyond single-node k3d to a multi-AZ cluster with `PodDisruptionBudgets (minAvailable: 1)` and anti-affinity to protect against node crashes.
4. **eBPF Security Policies:** Replace rigid stateless NACLs with Cilium eBPF to enforce stateful L7 filtering without exposing broad ephemeral port ranges.

6. Concluding Reflection

This project demonstrated how knowledge distillation and cloud-native orchestration bridge the gap between heavy AI models and resource-constrained environments, delivering resilient, low-latency deployment on budget hardware.

Submission Tag: final **Verified By:** Nemapu Navaneeth (Roll No: 230101086)